

```
In [67]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
In [68]: # Step 2: Generate Data

np.random.seed(0)

n_samples = 250 #(per class it is 250 so total 500 samples)

# Class 0
mu1 = [2, 2]
sigma1 = [[0.9, -0.0255],
          [-0.0255, 0.9]]
class0 = np.random.multivariate_normal(mu1, sigma1, n_samples)

# Class 1
mu2 = [5, 5]
sigma2 = [[0.5, 0],
          [0, 0.3]]
class1 = np.random.multivariate_normal(mu2, sigma2, n_samples)

# Labels
y0 = np.zeros(n_samples)
y1 = np.ones(n_samples)
```

```
In [69]: # Step 3: Combine Data

X = np.vstack((class0, class1))
y = np.hstack((y0, y1))
```

```
In [70]: # Step 4: Add Bias Term

bias = np.ones((X.shape[0], 1))
X = np.hstack((bias, X))
```

```
In [71]: # Step 5: Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
In [72]: # Step 6: Activation Function

def step_function(z):
    return 1 if z >= 0 else 0
```

```
In [73]: # Step 7: Perceptron Training

def perceptron_train(X, y, lr=0.1, max_epochs=1000):
    weights = np.zeros(X.shape[1])

    for epoch in range(max_epochs):
```

```

errors = 0

for i in range(X.shape[0]):
    z = np.dot(X[i], weights)
    y_pred = step_function(z)

    error = y[i] - y_pred

    if error != 0:
        weights += lr * error * X[i]
        errors += 1

    if errors == 0:
        print(f"Converged at epoch {epoch+1}")
        break

return weights

# Step 8: Train Model

weights = perceptron_train(X_train, y_train)

print("Learned Weights:", weights)

```

Converged at epoch 3
 Learned Weights: [-3.3 0.40638285 0.47899128]

```

In [74]: # Step 9: Prediction

def predict(X, weights):
    return np.array([step_function(np.dot(x, weights)) for x in X])

y_pred = predict(X_test, weights)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Step 10: Decision Boundary
plt.figure(figsize=(7,6))

plt.scatter(class0[:,0], class0[:,1], label="Class 0", alpha=0.6)
plt.scatter(class1[:,0], class1[:,1], label="Class 1", alpha=0.6)

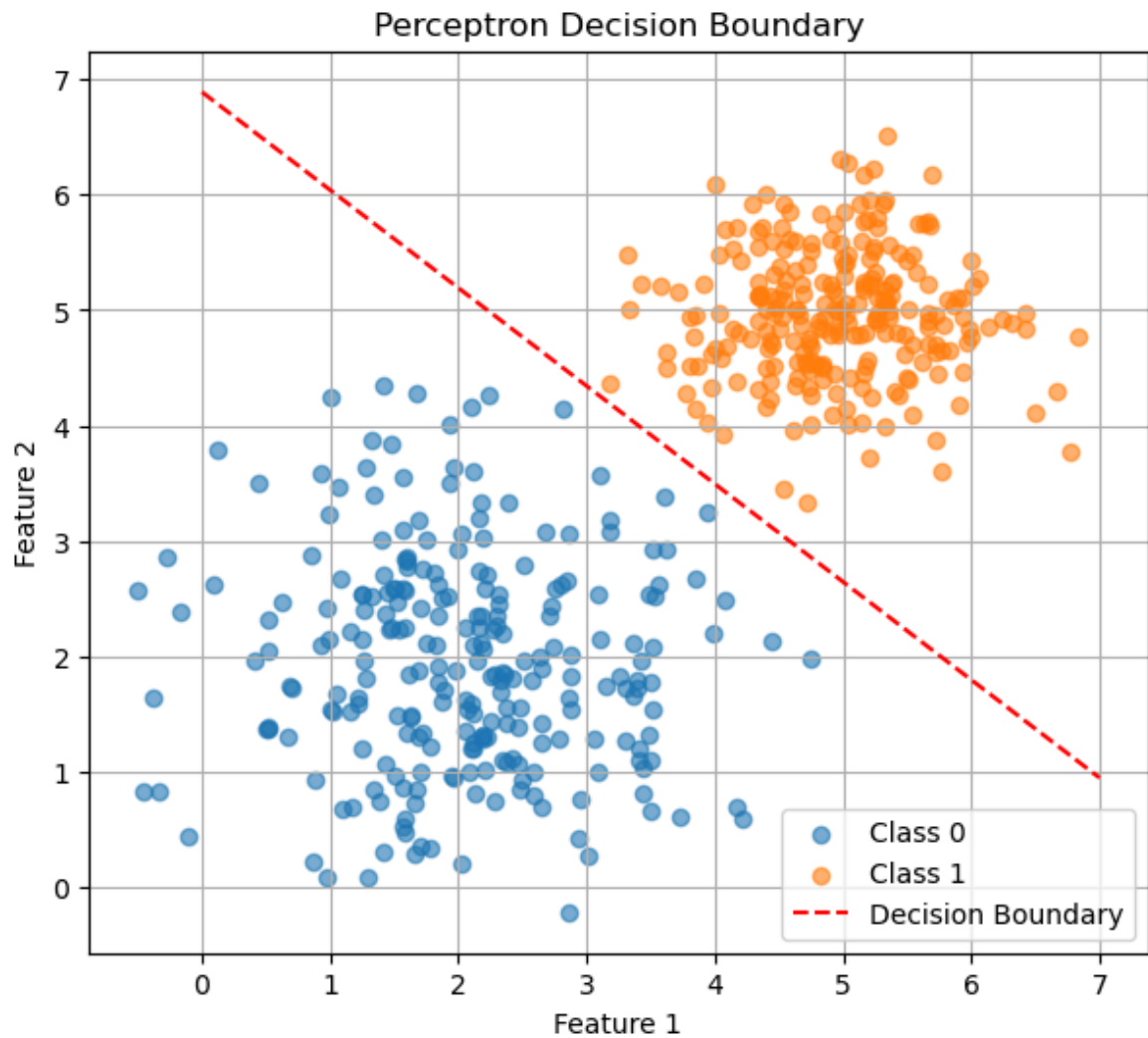
x_vals = np.linspace(0, 7, 100)

if abs(weights[2]) > 1e-9:
    y_vals = -(weights[0] + weights[1]*x_vals) / weights[2]
    plt.plot(x_vals, y_vals, 'r--', label="Decision Boundary")

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Perceptron Decision Boundary")
plt.legend()
plt.grid()
plt.show()

```

Accuracy: 1.0

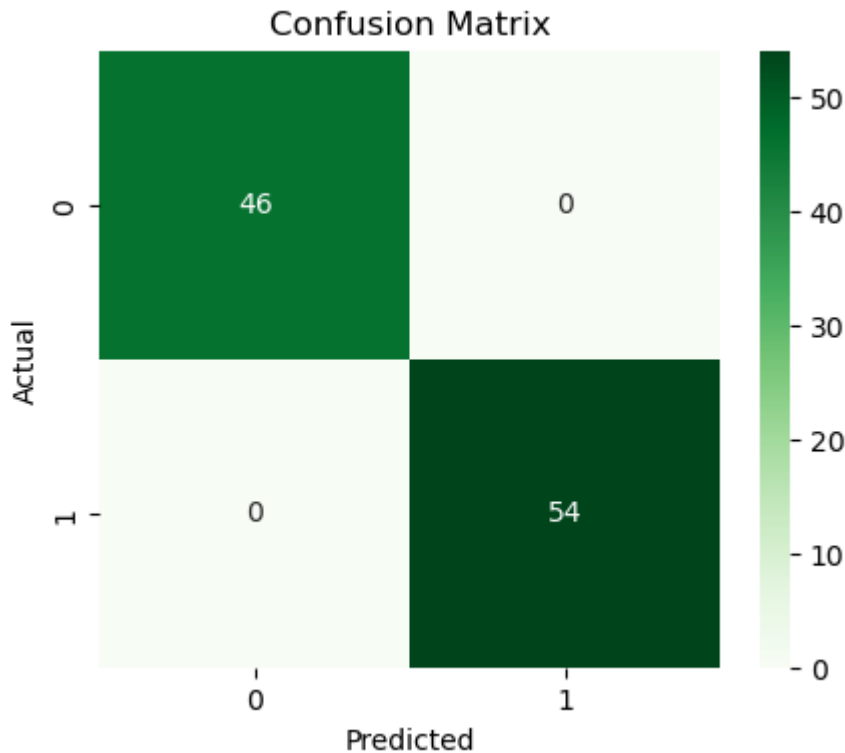


```
In [75]: cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

print("Confusion Matrix:\n", cm)
```



Confusion Matrix:
[[46 0]
[0 54]]

Report:

1:What are the parallels between Biological NNs and Artificial NNs? List any three.

Ans:Both are composed of interconnected processing units (neurons) that process information collectively. In both systems, connections determine signal strength (synapses in biological networks, weights in artificial networks). Both systems learn and adapt over time by adjusting connection strengths (synaptic plasticity vs weight updates).

2. What was the McCulloch-Pitts Neuron model? List any two of its limitations.

Ans:The McCulloch-Pitts Neuron Model is a simple binary threshold model that outputs 1 if the weighted sum of inputs exceeds a threshold, otherwise 0. Limitations (any two): Cannot solve non-linearly separable problems (e.g., XOR). Weights are fixed → no learning capability.

3. How does a perceptron learn?

Ans: A perceptron learns by repeatedly updating its weights based on the error between the predicted output and the actual output. For each training example, the weights are adjusted using a learning rate to reduce this error. This process continues until the model correctly classifies the data or a maximum number of iterations is reached.

4. List any three activation functions and their respective output ranges.

Ans: Three activation functions and their respective output ranges are as follows:

Step Function → Output: {0, 1}—either 0 or 1 Sigmoid Function → Output: (0, 1) ReLU (Rectified Linear Unit) → Output: $(0, \infty)$

5. Why was the classical perceptron not able to solve the XOR problem? Can you suggest a way to solve the XOR problem using neural networks but without going into a higher dimensional space?

Ans: The classical perceptron cannot solve the XOR problem because: XOR is not linearly separable ; A single linear boundary cannot separate the classes